

```

// EMERGENCE OS 1 - Input Modifier
const modifier = (text) => {
  try {
    EmergenceEngine.init();
    if (typeof EmergenceEngine.ensureConfigCard === 'function') {
      EmergenceEngine.ensureConfigCard();
    }

    const safeText = text || "";

    // Aggressive regex to rip the slash command out of any wrappers.
    // undercurrents/drives/threads all route to the same handler (aliases),
    // as do reflections/thoughts.
    const cmdMatch = safeText.match(/(?:\/) (help|settings|locations|loc|cleanup|f

if (cmdMatch) {
  let parsedCmd = "/" + cmdMatch[1].toLowerCase();
  if (cmdMatch[2]) {
    parsedCmd += " " + cmdMatch[2].trim();
  }

  const cmdResult = EmergenceEngine.processCommand(parsedCmd);

  if (cmdResult) {
    state.emergence.commandOutput = cmdResult;
    state.emergence.isCommandTurn = true;
    // Overwrite the input so the AI doesn't think you spoke
    text = "\n> [System Command Executed - Inspecting World State]";
  }
} else if (/^\s*\/[a-zA-Z]/.test(safeText)) {
  // Looks like an attempted command (starts with / followed by a letter)
  // that didn't match anything known - a typo, or a command that doesn't
  // exist. Give clear feedback instead of silently sending it to the AI as
  // a normal action, which would otherwise produce a confusing improvised
  // response to a command nobody meant as a story beat.
  state.emergence.commandOutput = "🤖 Command not recognized. Type /help to s
  state.emergence.isCommandTurn = true;
  text = "\n> [System Command Executed - Inspecting World State]";
} else {
  state.emergence.isCommandTurn = false;
  // Locations first: a specific "in/at/entering X" match is a more confident
  // signal than the bare-capitalized-word character heuristic, so letting
  // locations claim a name first stops places being misread as NPCs.
  EmergenceEngine.discoverLocations(safeText);

```

```
        if (state.emergence.config.HumanAgency === "Enabled") {
            EmergenceEngine.discoverCharacters(safeText);
        }
    }
    return { text: text || safeText || "\u200B" };
} catch (error) {
    return { text: text || "\u200B" };
}
};
modifier(text);
```